

Keep Your Free Backend Awake Using UptimeRobot

If you're using **any Cloud Platform's Free Plan**, your backend may go to sleep after being idle for some time. This can cause the first request to take much longer.

A simple way to reduce this is by using **UptimeRobot**, which periodically sends a request to your backend's health endpoint.

Note: This doesn't guarantee your backend will never sleep. It simply keeps it active by making regular requests.

Step 1: Create a Health Check Route

Open your backend project.

Express (JavaScript)

```
app.get("/health", (req, res) => {  
  res.status(200).json({  
    success: true,  
    message: "Server is healthy",  
    timestamp: new Date().toISOString(),  
  });  
});
```

Express (TypeScript)

```
app.get("/health", (_req, res) => {  
  res.status(200).json({  
    success: true,  
    message: "Server is healthy",  
    timestamp: new Date().toISOString(),  
  });  
});
```

Test the Route

After deploying your backend, open:

<https://your-backend.onrender.com/health>

You should see something like:

```
{
  "success": true,
  "message": "Server is healthy",
  "timestamp": "2026-07-31T12:34:56.000Z"
}
```

If this works, you're ready for the next step.

Step 2: Create a Free UptimeRobot Account

Visit:

<https://uptimerobot.com>

1. Click **Sign Up**.
 2. Create a free account.
 3. Verify your email.
 4. Log in to your dashboard.
-

Step 3: Create a New Monitor

Click the **+ Add New Monitor** button.

Select:

- **Monitor Type:** HTTP(s)
- **Friendly Name:** Any name you like (e.g., Backend Health)
- **URL (or IP):** Your deployed health route

Example:

<https://your-backend.onrender.com/health>

Step 4: Configure the Monitor

Recommended settings:

- **Monitoring Interval:** 5 Minutes (Free Plan)
- Leave the remaining settings as default.

Click:

Create Monitor

That's it 🎉

Step 5: Verify Everything Works

After a few minutes:

- Refresh your UptimeRobot dashboard.
- You should see the monitor status change to **Up**.

From now on, UptimeRobot will send a request to your `/health` endpoint every 5 minutes.

Why Does This Help?

Normally, a backend with no incoming traffic can become idle.

Since UptimeRobot keeps requesting your health endpoint, your backend receives regular traffic and is less likely to become idle.

⚠ Important: Don't Create Cron Jobs for Every Backend

Many cloud platforms have a **monthly usage limit** on their free plans.

For example, some providers offer around **750 free compute hours per month** (this can change depending on the platform and plan).

Since UptimeRobot keeps sending requests every few minutes, your backend stays active more often and may consume those free hours faster.

My Recommendation

If you're using multiple backend projects under the **same cloud account**, don't create a cron job for every single backend.

Instead:

-  Keep the cron monitor only for the backend you're actively developing or demonstrating.
-  Let older or inactive projects sleep naturally.
-  Remove monitors for projects you no longer need.

This helps you stay within your free plan limits while keeping your primary backend responsive.

Remember: Free plan limits and sleep behavior vary between cloud providers (and may change over time), so always check your provider's latest documentation before relying on this approach.

Frequently Asked Questions

Will this completely prevent Cloud Platform Server from sleeping?

No.

It helps keep your application active by sending periodic requests, but it's not an official Cloud Platform feature or guarantee.

Does this cost anything?

No.

The UptimeRobot free plan is enough for a single backend.

Can I monitor multiple backends?

Yes.

You can create multiple monitors for different backend URLs.

Does the health route need to connect to the database?

No.

Keep it lightweight.

It should only return a simple success response.

Example Folder Structure

```
src
├── routes
├── controllers
├── middleware
├── app.ts
└── server.ts
```

You can place the health route wherever you define your other Express routes.

If this guide helped you, feel free to share it with someone whose Render backend keeps sleeping. 🚀